

# UA Attendance – Line by Line Code Reference

Every line of the code, with its use – tabular format. Open [REFERENCE.pdf](#) from this file.

FILE: C:\inetpub\wwwroot\ua\app.py

## Lines 1–19 — Imports & Flask app config

| Line | Code                                                                                                               | Its use                                                         |
|------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| 1    | import os                                                                                                          | Access environment variables and file paths.                    |
| 2    | import json                                                                                                        | Read/write the users.json file.                                 |
| 3    | import threading                                                                                                   | Run the background sync thread.                                 |
| 4    | import time                                                                                                        | Pause 3s between sync cycles.                                   |
| 5    | from datetime import date, datetime                                                                                | Get today's date and current time stamps.                       |
| 6    | import pyodbc                                                                                                      | Connect to SQL Server.                                          |
| 7    | from flask import Flask, render_template_string, jsonify, request, redirect, session, url_for, send_from_directory | Flask web framework tools (templates, JSON, sessions).          |
| 8    | from flask_cors import CORS                                                                                        | Allow cross-origin requests.                                    |
| 9    | from functools import wraps                                                                                        | Helper for login decorators.                                    |
| 10   | from werkzeug.security import generate_password_hash, check_password_hash                                          | Password hashing helpers (imported, currently unused in logic). |
| 12   | app = Flask(__name__)                                                                                              | Create the Flask application object.                            |
| 13   | CORS(app)                                                                                                          | Enable CORS on all routes.                                      |
| 14   | app.secret_key = os.environ.get("SECRET_KEY", "ua-attendance-fixed-secret-key-2026")                               | Secret used to sign session cookies.                            |
| 15   | app.config.update(                                                                                                 | Start of app config block.                                      |
| 16   | SESSION_COOKIE_HTTPONLY=True,                                                                                      | Session cookie not readable by JavaScript (safer).              |
| 17   | SESSION_COOKIE_SAMESITE="Lax",                                                                                     | Cookie sent only on same-site navigation.                       |
| 18   | PERMANENT_SESSION_LIFETIME=480,                                                                                    | Session lifetime 480 minutes (8 hours).                         |
| 19   | )                                                                                                                  | End of config block.                                            |

## Lines 21–45 — Paths, DB config, locations

| Line | Code                                                  | Its use                                            |
|------|-------------------------------------------------------|----------------------------------------------------|
| 21   | BASE_DIR = os.path.dirname(os.path.abspath(__file__)) | Folder where app.py lives (C:\inetpub\wwwroot\ua). |
| 22   | USERS_FILE = os.path.join(BASE_DIR, "users.json")     | Full path to the users file.                       |
| 24   | DB_CONFIG = ( ... )                                   | SQL Server connection string (db eTimetracklite1). |
| 33   | LOG_TABLES = ["DeviceLogs_8_2026", ...]               | Monthly punch-log tables to scan for today.        |
| 35   | LIVE_CACHES = {}                                      | In-memory live attendance per device.              |
| 36   | USER_CACHE = None                                     | In-memory copy of users.json.                      |
| 37   | USERS_LOCK = threading.Lock()                         | Prevents two threads writing users.json at once.   |

|    |                                                                  |                                                     |
|----|------------------------------------------------------------------|-----------------------------------------------------|
| 39 | LOCATIONS = {                                                    | Start of device list.                               |
| 40 | 21: {"name": "UA_Neikarapatti", "location": "Neikarapatti"},     | Device 21 = Neikarapatti.                           |
| 41 | 23: {"name": "UA_Theevattipatti", "location": "Theevattipatti"}, | Device 23 = Theevattipatti.                         |
| 42 | 24: {"name": "UA_Sipcot", "location": "Sipcot"},                 | Device 24 = Sipcot (default display).               |
| 43 | 25: {"name": "UA_head_lab", "location": "Salem Head"},           | Device 25 = Salem Head lab.                         |
| 44 | 42: {"name": "UA_NAMAKKAL", "location": "Namakkal"},             | Device 42 = Namakkal.                               |
| 45 | }                                                                | End of device list. <b>Add/remove devices here.</b> |

## Lines 47–86 — Users file helpers & auth decorators

| Line | Code                                   | Its use                              |
|------|----------------------------------------|--------------------------------------|
| 47   | def load_users():                      | Define function to load users.       |
| 49   | with USERS_LOCK:                       | Lock while reading the file.         |
| 50   | if os.path.exists(USERS_FILE):         | If users.json exists...              |
| 51   | with open(USERS_FILE, "r") as f:       | Open it for reading.                 |
| 52   | USER_CACHE = json.load(f)              | Load JSON into cache.                |
| 54   | else: USER_CACHE = {}                  | If no file, use empty dict.          |
| 55   | return USER_CACHE                      | Return the users.                    |
| 57   | def save_users(users):                 | Define function to save users.       |
| 59   | with open(USERS_FILE, "w") as f:       | Open file for writing.               |
| 60   | json.dump(users, f, indent=2)          | Write JSON (pretty, 2-space indent). |
| 62   | USER_CACHE = dict(users)               | Refresh in-memory cache.             |
| 64   | load_users()                           | Load users once at startup.          |
| 66   | def login_required(f):                 | Decorator: requires a session user.  |
| 69   | if "user" not in session:              | If not logged in...                  |
| 70   | return redirect(url_for("login"))      | ...redirect to login page.           |
| 74   | def role_required(*roles):             | Decorator: require a specific role.  |
| 82   | if user_data.get("role") not in roles: | If role not allowed...               |
| 83   | return redirect(url_for("login"))      | ...redirect to login.                |

## Lines 88–225 — Core sync engine (runs every 3s)

| Line | Code                                                                                                                                                       | Its use                                                |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| 88   | <code>def database_sync_worker():</code>                                                                                                                   | Define the background worker.                          |
| 89   | <code>    while True:</code>                                                                                                                               | Loop forever.                                          |
| 91   | <code>        today = date.today()</code>                                                                                                                  | Get today's date.                                      |
| 92   | <code>        today_str = today.strftime("%Y-%m-%d")</code>                                                                                                | Format as YYYY-MM-DD.                                  |
| 93   | <code>        device_ids = list(LOCATIONS.keys())</code>                                                                                                   | List of all configured devices.                        |
| 95   | <code>        conn = pyodbc.connect(DB_CONFIG)</code>                                                                                                      | Open DB connection.                                    |
| 96   | <code>        cursor = conn.cursor()</code>                                                                                                                | Create DB cursor.                                      |
| 98   | <code>        cursor.execute(""" SELECT EmployeeCode, EmployeeName, DeviceId<br/>FROM Employees ... """)</code>                                            | Load all employees.                                    |
| 103  | <code>    all_emps = cursor.fetchall()</code>                                                                                                              | Fetch results.                                         |
| 105  | <code>    emp_map = {}</code>                                                                                                                              | Build global name map.                                 |
| 106  | <code>    for r in all_emps:</code>                                                                                                                        | Loop each employee row.                                |
| 107  | <code>        code = str(r[0]).strip()</code>                                                                                                              | Employee code.                                         |
| 108  | <code>        name = str(r[1]).strip() ...</code>                                                                                                          | Name or fallback "Employee {code}".                    |
| 110  | <code>        emp_map[code] = {"name": name, "device_id": dev_id}</code>                                                                                   | Store code → name/device.                              |
| 112  | <code>    device_emps = {}</code>                                                                                                                          | Map device → enrolled employees.                       |
| 115  | <code>    cursor.execute(f""" SELECT du.DeviceId, e.EmployeeCode,<br/>e.EmployeeName FROM DeviceUsers du LEFT JOIN Employees e ... """)</code>             | Which employees are enrolled per device.               |
| 121  | <code>    for r in cursor.fetchall():</code>                                                                                                               | Loop enrolled rows.                                    |
| 126  | <code>        device_emps.setdefault(dev, {})[code] = name</code>                                                                                          | Store enrollment under device.                         |
| 130  | <code>    logs_by_device = {}</code>                                                                                                                       | Today's punches grouped by device.                     |
| 137  | <code>    for table in LOG_TABLES:</code>                                                                                                                  | Try each log table.                                    |
| 143  | <code>        cursor.execute(f""" SELECT UserId, LogDate, C1, DeviceId FROM<br/>{table} WHERE DeviceId IN (...) AND CONVERT(date, LogDate) = ? """)</code> | Query today's punches from this table.                 |
| 147  | <code>        break</code>                                                                                                                                 | Stop after first table that works.                     |
| 151  | <code>    for row in all_logs:</code>                                                                                                                      | Loop each punch row.                                   |
| 152  | <code>        uid = str(row[0]).strip()</code>                                                                                                             | Employee/User id.                                      |
| 156  | <code>        logs_by_device[dev_id].append(...)</code>                                                                                                    | Add punch to that device.                              |
| 161  | <code>    for dev_id in device_ids:</code>                                                                                                                 | Process each device.                                   |
| 166  | <code>        enrolled = list(device_emps.get(dev_id, {}).keys())</code>                                                                                   | Employees enrolled on device.                          |
| 167  | <code>        assigned = [...]</code>                                                                                                                      | Employees whose Employees.DeviceId = this device.      |
| 168  | <code>        relevant = list(dict.fromkeys(enrolled + assigned +<br/>list(punch_users)))</code>                                                           | Final list = enrolled + assigned + anyone who punched. |
| 172  | <code>        employees = {}</code>                                                                                                                        | Build per-device employee dict.                        |
| 173  | <code>        for uid in relevant:</code>                                                                                                                  | For each employee...                                   |
| 174  | <code>            if uid in device_emps.get(dev_id, {}):</code>                                                                                            | If enrolled → use enrolled name.                       |
| 176  | <code>            elif uid in emp_map:</code>                                                                                                              | Else if in global employees...                         |
| 179  | <code>            else: name = f"Employee {uid}"</code>                                                                                                    | Else show "Employee {id}".                             |

|     |                                                                                                |                                                                       |
|-----|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| 180 | employees[uid] = {"name": name, "punches": [], "status": "Absent", "last_punch": "---:--:--"}  | Create employee record, default Absent.                               |
| 186 | for uid, emp in employees.items():                                                             | Compute status for each.                                              |
| 187 | if emp["punches"]:                                                                             | If employee has punches today...                                      |
| 191 | emp["status"] = "Present" if last["dir"] == "in" or len(emp["punches"]) % 2 == 1 else "Absent" | <b>KEY RULE:</b> Present if last punch is "in" OR punch count is odd. |
| 199 | present = sum(...)                                                                             | Count Present.                                                        |
| 200 | absent = sum(...)                                                                              | Count Absent.                                                         |
| 206 | LIVE_CACHES[dev_id] = {...}                                                                    | Save result to cache.                                                 |
| 211 | "device_status": {...}                                                                         | Device name/location/last_punch/status.                               |
| 215 | "status": "Online" if last_punch else "Offline",                                               | Online if any punch today, else Offline.                              |
| 217 | "last_updated": datetime.now().strftime("%I:%M:%S %p")                                         | Time of last cache update.                                            |
| 220 | conn.close()                                                                                   | Close DB connection.                                                  |
| 222 | except Exception as e:                                                                         | Catch any error.                                                      |
| 223 | print(f"Sync error: {e}")                                                                      | Print the error to log.                                               |
| 225 | time.sleep(3)                                                                                  | <b>Wait 3 seconds, then repeat.</b>                                   |

## Lines 596–705 — Routes (API used by React)

| Line | Code                                                                                         | Its use                                          |
|------|----------------------------------------------------------------------------------------------|--------------------------------------------------|
| 596  | <code>FRONTEND_DIR = os.path.join(BASE_DIR, "dist")</code>                                   | Where the built React app lives.                 |
| 598  | <code>@app.route("/", defaults={"path": ""})</code>                                          | Route for site root.                             |
| 599  | <code>@app.route("/&lt;path:path&gt;")</code>                                                | Route for any path under /.                      |
| 600  | <code>def serve_frontend(path):</code>                                                       | Serve the SPA.                                   |
| 602  | <code>if path and os.path.isfile(full):</code>                                               | If file exists in dist...                        |
| 603  | <code>return send_from_directory(FRONTEND_DIR, path)</code>                                  | ...send it.                                      |
| 604  | <code>return send_from_directory(FRONTEND_DIR, "index.html")</code>                          | Else fall back to index.html (SPA routing).      |
| 683  | <code>@app.route("/api/login", methods=["POST"])</code>                                      | Login API endpoint.                              |
| 685  | <code>data = request.get_json(force=True, silent=True) or {}</code>                          | Read JSON body.                                  |
| 686  | <code>username = (data.get("username") or "").strip()</code>                                 | Read username.                                   |
| 687  | <code>password = (data.get("password") or "").strip()</code>                                 | Read password.                                   |
| 689  | <code>user_data = users.get(username)</code>                                                 | Look up user.                                    |
| 690  | <code>if user_data and user_data.get("password") == password:</code>                         | If credentials match...                          |
| 691  | <code>return jsonify({...})</code>                                                           | ...return success + role + device_id.            |
| 698  | <code>return jsonify({"success": False, ...}), 401</code>                                    | Else return 401 error.                           |
| 700  | <code>@app.route("/api/locations")</code>                                                    | Locations API.                                   |
| 702  | <code>return jsonify([{"device_id": did, ...} for did, loc in<br/>LOCATIONS.items()])</code> | Return all devices as JSON list.                 |
| 655  | <code>@app.route("/api/live/&lt;int:device_id&gt;")</code>                                   | Live data for one device.                        |
| 657  | <code>data = LIVE_CACHES.get(device_id)</code>                                               | Read from cache.                                 |
| 658  | <code>if not data:</code>                                                                    | If not cached...                                 |
| 659  | <code>return jsonify({"present":0,...})</code>                                               | ...return zeros.                                 |
| 663  | <code>api_status = "IN" if db_status == "Present" else "OUT"</code>                          | Convert Present → IN, Absent → OUT.              |
| 673  | <code>return jsonify({...})</code>                                                           | Return<br>present/absent/employees/deviceStatus. |

## Lines 715–873 — User APIs & background start

| Line | Code                                                                       | Its use                                   |
|------|----------------------------------------------------------------------------|-------------------------------------------|
| 707  | <code>API_KEY = "admin123"</code>                                          | Static key for user-management endpoints. |
| 715  | <code>@app.route("/api/users", methods=["GET", "POST"])</code>             | List/Create users.                        |
| 722  | <code>if request.method == "GET":</code>                                   | GET → list users (excluding superadmin).  |
| 735  | <code>if request.method == "POST":</code>                                  | POST → create user.                       |
| 743  | <code>if not username or not password or not label:</code>                 | Validation.                               |
| 745  | <code>if username in users:</code>                                         | Duplicate check.                          |
| 748  | <code>users[username] = {...}</code>                                       | Add new user.                             |
| 755  | <code>save_users(users)</code>                                             | Save to users.json.                       |
| 758  | <code>@app.route("/api/users/&lt;username&gt;", methods=["DELETE"])</code> | Delete a user.                            |

|     |                                                                                       |                                  |
|-----|---------------------------------------------------------------------------------------|----------------------------------|
| 764 | <code>if username not in users or users[username].get("role") == "superadmin":</code> | Cannot delete superadmin.        |
| 766 | <code>del users[username]</code>                                                      | Delete user.                     |
| 865 | <code>_THREAD_STARTED = False</code>                                                  | Flag so thread starts only once. |
| 870 | <code>t = threading.Thread(target=database_sync_worker, daemon=True)</code>           | Create background sync thread.   |
| 871 | <code>t.start()</code>                                                                | Start it (runs forever).         |

## FRONTEND SOURCE FILES

### FILE: src\App.tsx – Session & routing

| Line | Code                                                         | Its use                                    |
|------|--------------------------------------------------------------|--------------------------------------------|
| 1    | import { useState } from 'react';                            | React state hook.                          |
| 2    | import LoginPage from './components/LoginPage';              | Login screen component.                    |
| 3    | import Dashboard from './components/Dashboard';              | Main dashboard component.                  |
| 4    | import AdminPage from './components/AdminPage';              | Admin panel component.                     |
| 5    | import SidebarLayout from './components/SidebarLayout';      | App shell component.                       |
| 7    | type Page = 'login'   'admin';                               | Which page is visible.                     |
| 9    | const STORAGE_KEY = 'ua_session_user';                       | localStorage key for session persistence.  |
| 11   | function loadUser() {                                        | Read saved user from localStorage.         |
| 12   | try { const raw = localStorage.getItem(STORAGE_KEY);         | Get stored JSON.                           |
| 13   | if (!raw) return null;                                       | Nothing stored → no user.                  |
| 14   | const u = JSON.parse(raw);                                   | Parse it.                                  |
| 15   | return u && u.username ? u : null;                           | Valid if it has a username.                |
| 17   | catch { return null; }                                       | On error, no user.                         |
| 20   | const [page, setPage] = useState<Page>('login');             | Current page state.                        |
| 21   | const [user, setUser] = useState<...>(loadUser);             | User state, initialized from localStorage. |
| 24   | function handleLogin(username, role, label, deviceId) {      | Called after successful login.             |
| 25   | const next = { username, role, label, deviceId };            | Build user object.                         |
| 26   | setUser(next);                                               | Update state.                              |
| 27   | localStorage.setItem(STORAGE_KEY, JSON.stringify(next));     | <b>Save session → survives refresh.</b>    |
| 31   | function handleLogout() {                                    | Logout handler.                            |
| 32   | setUser(null);                                               | Clear state.                               |
| 33   | localStorage.removeItem(STORAGE_KEY);                        | Delete saved session.                      |
| 36   | if (!user) return <LoginPage onLogin={handleLogin} />;       | No user → show login.                      |
| 39   | if (page === 'admin') return <AdminPage ... />;              | Admin page when requested.                 |
| 49   | return (<SidebarLayout ...><Dashboard .../></SidebarLayout>) | Default: shell with dashboard.             |

### FILE: src\data\mockData.ts – Config values

| Line | Code                                                                  | Its use                             |
|------|-----------------------------------------------------------------------|-------------------------------------|
| 1    | export const API_URL = import.meta.env.BASE_URL.replace(/\/+\$/, ''); | API base = /UA (same origin).       |
| 2    | export const DEVICE_ID = 24;                                          | Default device if manager has none. |

### FILE: src\types.ts – Type definitions

| Line | Code                               | Its use                              |
|------|------------------------------------|--------------------------------------|
| 1    | export interface PunchRecord {...} | Single punch (time + direction).     |
| 6    | export interface Employee {...}    | Employee: id, name, status, punches. |

|    |                                               |                           |
|----|-----------------------------------------------|---------------------------|
| 16 | <code>export interface LiveData {...}</code>  | Live API response shape.  |
| 26 | <code>export type DetailedStatus = ...</code> | Possible detail statuses. |

#### FILE: `src\data\attendanceUtils.ts` – Shift rules

| Line | Code                                                 | Its use                               |
|------|------------------------------------------------------|---------------------------------------|
| 26   | <code>const startMins = 8 * 60 + 30;</code>          | Shift start 8:30 AM.                  |
| 27   | <code>const lateMins = 9 * 60 + 30;</code>           | Late threshold 9:30 AM.               |
| 28   | <code>const overtimeMins = 18 * 60;</code>           | Overtime threshold 6:00 PM.           |
| 19   | <code>export function getDetailedStatus(emp)</code>  | Return 'working'/'late'/'absent'/etc. |
| 55   | <code>export function getStatusConfig(status)</code> | Chip color/label per status.          |
| 68   | <code>export function calcHoursToday(emp)</code>     | Hours worked today.                   |

## FILE: src\components\Dashboard.tsx – Live view

| Line | Code                                                                                            | Its use                                   |
|------|-------------------------------------------------------------------------------------------------|-------------------------------------------|
| 1    | import { useEffect, useState, useMemo } from 'react';                                           | React hooks.                              |
| 2    | import EmployeeCard from '../EmployeeCard';                                                     | Employee card component.                  |
| 3    | import { API_URL, DEVICE_ID } from '../data/mockData';                                          | Config values.                            |
| 5    | import { getDetailedStatus } from '../data/attendanceUtils';                                    | Filtering by detail status.               |
| 21   | const FILTER_OPTIONS = [...]                                                                    | Filter dropdown items.                    |
| 31   | export default function Dashboard({role, deviceId, statusFilter, onFilterChange})               | Component entry.                          |
| 32   | const [data, setData] = useState<LiveData   null>(null);                                        | Live data state.                          |
| 38   | const canSwitchLocation = role === 'superadmin'    role === 'admin';                            | Only admin/superadmin can switch devices. |
| 40   | async function fetchLocations()                                                                 | Load /api/locations.                      |
| 50   | async function fetchData()                                                                      | Load /api/live/<device>.                  |
| 51   | const dev = activeDevice ?? DEVICE_ID;                                                          | Use chosen device or default 24.          |
| 62   | useEffect(() => { fetchData(); const id = setInterval(fetchData, 2000); ... }, [activeDevice]); | <b>Refresh every 2 seconds.</b>           |
| 68   | const sorted = useMemo(...)                                                                     | Sort employees by first punch time.       |
| 83   | const filtered = useMemo(...)                                                                   | Apply status filter.                      |
| 91   | <div className="bg-white border-b ..." >                                                        | Header bar.                               |
| 97   | <h1>{data ? data.deviceName : 'UA Attendance'}</h1>                                             | Device name.                              |
| 101  | {canSwitchLocation && ...}                                                                      | Location switcher dropdown.               |
| 133  | <span ...>ONLINE/OFFLINE</span>                                                                 | Device connectivity badge (green/red).    |
| 141  | {data.present} IN / {data.absent} OUT                                                           | Summary in header.                        |
| 157  | <div className="grid grid-cols-3 ..." > Present/Absent/Total                                    | Summary cards.                            |
| 200  | grid ... {filtered.map((emp, i) => <EmployeeCard .../>)}                                        | Employee cards grid.                      |

## FILE: src\components\EmployeeCard.tsx – One employee

| Line | Code                                                                         | Its use                   |
|------|------------------------------------------------------------------------------|---------------------------|
| 19   | const hasPunch = Boolean(employee.lastPunchRaw);                             | Did this person punch?    |
| 20   | const [seconds, setSeconds] = useState(0);                                   | Live duration counter.    |
| 22   | const detailStatus = useMemo(() => getDetailedStatus(employee), [employee]); | Status label for chip.    |
| 26   | useEffect(...) { tick every 200ms }                                          | Update IN-duration live.  |
| 46   | <p>{employee.name}</p>                                                       | Employee name.            |
| 47   | status chip                                                                  | Working/Late/Absent pill. |
| 52   | ID: {employee.id}                                                            | Employee id.              |
| 56   | In/Last: {employee.lastPunch}                                                | Punch time.               |
| 61   | Dur: {formatDuration(seconds)}                                               | Duration in/out.          |

**FILE: src\components\AdminPage.tsx – Admin panel**

| Line | Code                                                                          | Its use                                 |
|------|-------------------------------------------------------------------------------|-----------------------------------------|
| 55   | <code>const canManageAll = role === 'superadmin'    role === 'admin';</code>  | Can see all devices.                    |
| 60   | <code>async function fetchUsers() { fetch('/api/users?key=admin123') }</code> | Load user list.                         |
| 68   | <code>async function fetchLocations()</code>                                  | Load devices.                           |
| 79   | <code>async function fetchLiveSummaries()</code>                              | Load live data for all allowed devices. |
| 96   | <code>useEffect(..., 5000)</code>                                             | Refresh summaries every 5s.             |
| 106  | <code>async function handleCreate(e)</code>                                   | Create user via POST /api/users.        |
| 131  | <code>async function handleDelete(username)</code>                            | Delete user via DELETE /api/users.      |
| 190  | <code>&lt;div&gt; All Locations cards</code>                                  | Per-device live overview.               |
| 226  | <code>Add User form</code>                                                    | Create manager/admin.                   |
| 274  | <code>Users table + Delete</code>                                             | Manage users.                           |

**FILE: src\components>LoginPage.tsx – Login form**

| Line | Code                                                                                                     | Its use                              |
|------|----------------------------------------------------------------------------------------------------------|--------------------------------------|
| 19   | <code>useEffect(() =&gt; { setUsername(''); setPassword(''); }, []);</code>                              | Clear fields on mount (no autofill). |
| 24   | <code>function handleSubmit(e)</code>                                                                    | On form submit.                      |
| 28   | <code>fetch(`\${API_URL}/api/login`, {method:'POST', body: JSON.stringify({username, password})})</code> | Call login API.                      |
| 35   | <code>if (r.ok &amp;&amp; d.success) onLogin(d.username, d.role, d.label, d.device_id)</code>            | On success → enter app.              |
| 38   | <code>else setError('Invalid username or password')</code>                                               | Show error.                          |

## users.json – Account store

| Field          | Example                            | Its use                         |
|----------------|------------------------------------|---------------------------------|
| username (key) | "admin_sipcot"                     | Login username.                 |
| password       | "Admin@sipcot"                     | Login password (plain text).    |
| role           | "superadmin" / "admin" / "manager" | Permission level.               |
| label          | "Admin - UA_Sipcot"                | Display name shown in UI.       |
| device_id      | 24                                 | Which device this user manages. |
| location       | "Sipcot"                           | Display location.               |

### Quick task lookup

| Task                                 | File / place                  |
|--------------------------------------|-------------------------------|
| Add or remove a device               | LOCATIONS in app.py (L39-45)  |
| Change device name/location shown    | app.py LOCATIONS + users.json |
| Change shift start / late / overtime | attendanceUtils.ts L26-28     |
| Change default device                | mockData.ts DEVICE_ID         |
| Change auto-refresh speed            | Dashboard.tsx L64 (2000)      |
| Change login persistence key         | App.tsx STORAGE_KEY           |
| Add/delete a user                    | users.json OR Admin Panel     |

### Deploy steps

| Step           | Action                                 |
|----------------|----------------------------------------|
| Build frontend | npm run build in attendance-dashboard\ |
| Copy to server | dist\ → C:\inetpub\wwwroot\ua\dist\    |
| Backend change | edit app.py / users.json directly      |
| Apply          | recycle IIS DefaultAppPool if needed   |